

NLP (Natural Language Processing) Interview Questions & Answers Cheat Sheet

Table of Contents

1. [NLP Fundamentals](#)
2. [Text Preprocessing](#)
3. [Feature Extraction Techniques](#)
4. [Word Embeddings](#)
5. [Language Models](#)
6. [Named Entity Recognition \(NER\)](#)
7. [Sentiment Analysis](#)
8. [Text Classification](#)
9. [Machine Translation](#)
10. [Advanced NLP Concepts](#)
11. [Transformer Architecture](#)
12. [BERT and Modern Models](#)
13. [Evaluation Metrics](#)
14. [Practical Implementation](#)

NLP Fundamentals

Q1: What is Natural Language Processing (NLP)?

Answer:

NLP is a subfield of artificial intelligence that focuses on enabling computers to understand, interpret, and generate human language in a meaningful way.

Key Components:

- **Computational Linguistics:** Understanding language structure
- **Machine Learning:** Learning patterns from data
- **Deep Learning:** Neural networks for complex language tasks
- **Statistics:** Probabilistic models for language

Applications:

- **Machine Translation:** Google Translate, DeepL

- **Chatbots:** Customer service, virtual assistants
- **Sentiment Analysis:** Social media monitoring, reviews
- **Information Extraction:** Document analysis, search engines
- **Text Summarization:** News articles, research papers
- **Question Answering:** Search engines, virtual assistants

Q2: What are the main challenges in NLP?

Answer:

1. Ambiguity:

- **Lexical:** "Bank" can mean financial institution or river bank
- **Syntactic:** "I saw the man with the telescope"
- **Semantic:** "The chicken is ready to eat"
- **Pragmatic:** Context-dependent meanings

2. Language Complexity:

- **Polysemy:** One word, multiple meanings
- **Synonymy:** Multiple words, same meaning
- **Homonymy:** Same spelling, different meanings
- **Irony and Sarcasm:** Figurative language

3. Context Dependency:

- **Anaphora:** "John went to the store. He bought milk."
- **Cataphora:** "When he arrived, John was tired."
- **Discourse:** Long-range dependencies

4. Variation and Change:

- **Dialects:** Regional language differences
- **Informal Language:** Slang, abbreviations, emojis
- **Language Evolution:** New words and meanings

Q3: What are the different levels of NLP?

Answer:

1. Phonological Analysis:

- Sound patterns in language
- Speech recognition and synthesis

2. Morphological Analysis:

- Word structure and formation
- Stemming and lemmatization

3. Lexical Analysis:

- Tokenization and vocabulary
- Part-of-speech tagging

4. Syntactic Analysis (Parsing):

- Sentence structure and grammar
- Dependency and constituency parsing

5. Semantic Analysis:

- Word and sentence meaning
- Semantic role labeling

6. Pragmatic Analysis:

- Context and intended meaning
- Discourse analysis

7. Discourse Analysis:

- Text structure and coherence
- Anaphora resolution

Text Preprocessing

Q4: What is tokenization and what are its types?

Answer:

Tokenization: Process of breaking text into smaller units (tokens) for analysis.

Types:

1. Word Tokenization:

```
import nltk
from nltk.tokenize import word_tokenize

text = "Natural Language Processing is fascinating!"
tokens = word_tokenize(text)
print(tokens)
# Output: ['Natural', 'Language', 'Processing', 'is', 'fascinating', '!']
```

2. Sentence Tokenization:

```

from nltk.tokenize import sent_tokenize

text = "NLP is great. It has many applications. Let's learn more!"
sentences = sent_tokenize(text)
print(sentences)
# Output: ['NLP is great.', 'It has many applications.', "Let's learn more!"]

```

3. Subword Tokenization:

- **Byte Pair Encoding (BPE):** Used in GPT models
- **WordPiece:** Used in BERT
- **SentencePiece:** Language-agnostic tokenization

Challenges:

- **Contractions:** "don't" → ["do", "n't"] or ["don't"]
- **Punctuation:** Attached or separate tokens
- **Hyphenated words:** "state-of-the-art"
- **URLs and emails:** Special handling required

Q5: Explain stemming and lemmatization. What are the differences?

Answer:

Aspect	Stemming	Lemmatization
Purpose	Remove suffixes to get root	Reduce to dictionary form
Output	May not be valid word	Always valid word
Context	Ignores context	Considers part of speech
Speed	Faster	Slower
Accuracy	Lower	Higher

Stemming Example:

```

from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
words = ["running", "runs", "ran", "runner"]
for word in words:
    print(f"{word} → {stemmer.stem(word)}")

# Output:
# running → run
# runs → run
# ran → ran
# runner → runner

```

Lemmatization Example:

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet

lemmatizer = WordNetLemmatizer()

# Verb lemmatization
print(lemmatizer.lemmatize("running", pos="v")) # run
print(lemmatizer.lemmatize("better", pos="a"))   # good

# Context matters
print(lemmatizer.lemmatize("meeting", pos="n"))  # meeting (noun)
print(lemmatizer.lemmatize("meeting", pos="v"))  # meet (verb)
```

When to Use:

- **Stemming:** Information retrieval, search engines (speed matters)
- **Lemmatization:** Text classification, sentiment analysis (accuracy matters)

Q6: What are stop words and how do you handle them?

Answer:

Stop Words: Common words that carry little semantic meaning (the, is, at, which, on).

Reasons to Remove:

- Reduce noise in text analysis
- Decrease computational complexity
- Focus on meaningful words
- Improve model performance

Example:

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('english'))
text = "This is a sample sentence demonstrating stop words."

tokens = word_tokenize(text)
filtered_tokens = [w for w in tokens if w.lower() not in stop_words]

print("Original:", tokens)
print("Filtered:", filtered_tokens)
# Original: ['This', 'is', 'a', 'sample', 'sentence', 'demonstrating', 'stop', 'words', '.']
# Filtered: ['sample', 'sentence', 'demonstrating', 'stop', 'words', '.']
```

Custom Stop Words:

```
# Add domain-specific stop words
custom_stop_words = stop_words.union({'said', 'say', 'get', 'go'})

# Remove stop words for specific tasks
# For sentiment analysis, keep negation words: 'not', 'no', 'never'
sentiment_stop_words = stop_words - {'not', 'no', 'never', 'nor', 'neither'}
```

When NOT to Remove:

- **Sentiment Analysis:** Negation words are important
- **Question Answering:** "What", "Where", "When" are crucial
- **Machine Translation:** All words may be needed for structure

Feature Extraction Techniques

Q7: Explain Bag of Words (BoW) model.

Answer:

Bag of Words: Text representation that ignores order and structure, focusing only on word frequency.

Process:

1. Create vocabulary from all documents
2. Represent each document as vector of word counts
3. Each dimension represents a unique word

Example:

```
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "I love machine learning",
    "Machine learning is fascinating",
    "I love programming"
]

vectorizer = CountVectorizer()
bow_matrix = vectorizer.fit_transform(documents)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("BoW Matrix:")
print(bow_matrix.toarray())

# Output:
# Vocabulary: ['fascinating' 'is' 'learning' 'love' 'machine' 'programming']
# BoW Matrix:
# [[0 0 1 1 1 0] # "I love machine learning"
```

```
# [1 1 1 0 1 0] # "Machine learning is fascinating"
# [0 0 0 1 0 1]] # "I love programming"
```

Advantages:

- Simple and interpretable
- Works well for classification
- Handles variable length documents

Disadvantages:

- Ignores word order ("not good" vs "good not")
- High dimensionality (sparse matrices)
- No semantic understanding
- Equal weight to all words

Q8: What is TF-IDF and how is it calculated?**Answer:**

TF-IDF (Term Frequency-Inverse Document Frequency): Weighting scheme that reflects word importance in document relative to collection.

Formula:

- **TF(t,d)** = (Number of times term t appears in document d) / (Total number of terms in document d)
- **IDF(t)** = $\log(\text{Total number of documents} / \text{Number of documents containing term t})$
- **TF-IDF(t,d)** = $\text{TF}(t,d) \times \text{IDF}(t)$

Intuition:

- **High TF:** Word appears frequently in document
- **High IDF:** Word is rare across collection (more informative)
- **High TF-IDF:** Word is frequent in document but rare overall

Implementation:

```
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "The cat sat on the mat",
    "The dog ran in the park",
    "Cats and dogs are pets"
]

tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(documents)
```

```
print("Features:", tfidf.get_feature_names_out())
print("TF-IDF Matrix:")
print(tfidf_matrix.toarray())

# Words like "the" get low scores (high frequency across documents)
# Words like "cat", "park" get higher scores (more discriminative)
```

Variations:

- **Log normalization:** $\log(1 + \text{tf})$
- **Sublinear scaling:** $1 + \log(\text{tf})$
- **L2 normalization:** Normalize document vectors to unit length

Word Embeddings

Q9: What are word embeddings and why are they important?

Answer:

Word Embeddings: Dense vector representations of words that capture semantic relationships in continuous space.

Key Properties:

- **Semantic Similarity:** Similar words have similar vectors
- **Analogies:** king - man + woman $\approx$ queen
- **Dimensionality:** Typically 100-300 dimensions
- **Continuous:** Real-valued vectors (not sparse)

Advantages over Traditional Methods:

- **Semantic Understanding:** Captures meaning relationships
- **Dimensionality Reduction:** Dense vs sparse representations
- **Generalization:** Similar words share properties
- **Transfer Learning:** Pre-trained embeddings available

Applications:

- Text classification
- Sentiment analysis
- Machine translation
- Information retrieval
- Recommendation systems

Q10: Explain Word2Vec algorithm and its variants.

Answer:

Word2Vec: Neural network approach to learn word embeddings by predicting context.

Two Architectures:

1. Continuous Bag of Words (CBOW):

- **Input:** Context words (surrounding words)
- **Output:** Target word (center word)
- **Process:** Average context embeddings to predict center word
- **Use Case:** Better for frequent words

2. Skip-gram:

- **Input:** Target word (center word)
- **Output:** Context words (surrounding words)
- **Process:** Use center word to predict surrounding words
- **Use Case:** Better for rare words, larger datasets

Training Process:

```
# Conceptual example
sentence = "The quick brown fox jumps over the lazy dog"
window_size = 2

# Skip-gram pairs (target → context)
# Target: "brown" → Context: ["The", "quick", "fox", "jumps"]
# Target: "fox" → Context: ["quick", "brown", "jumps", "over"]
```

Implementation:

```
from gensim.models import Word2Vec

# Sample sentences
sentences = [
    ["natural", "language", "processing"],
    ["machine", "learning", "algorithms"],
    ["deep", "learning", "neural", "networks"]
]

# Train Word2Vec model
model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, sg=1)

# Get word vector
vector = model.wv['language']
print(f"Vector shape: {vector.shape}")

# Find similar words
```

```
similar = model.wv.most_similar('language', topn=3)
print(f"Similar to 'language': {similar}")
```

Optimization Techniques:

- **Hierarchical Softmax:** Efficient for large vocabularies
- **Negative Sampling:** Sample negative examples instead of full softmax

Q11: What is the difference between Word2Vec, GloVe, and FastText?

Answer:

Aspect	Word2Vec	GloVe	FastText
Approach	Neural, local context	Matrix factorization, global	Extension of Word2Vec
Training	Predict context/target	Global co-occurrence matrix	Subword information
Speed	Moderate	Fast	Moderate
OOV Handling	No	No	Yes (subwords)
Memory	Moderate	High (matrix storage)	Higher (subword vectors)

GloVe (Global Vectors):

```
# GloVe combines global statistics with local context
# Pre-trained vectors available
import numpy as np

# Load pre-trained GloVe vectors
def load_glove_vectors(file_path):
    embeddings = {}
    with open(file_path, 'r', encoding='utf-8') as f:
        for line in f:
            values = line.split()
            word = values[0]
            vector = np.asarray(values[1:], dtype='float32')
            embeddings[word] = vector
    return embeddings
```

FastText:

- **Subword Information:** Breaks words into character n-grams
- **OOV Handling:** Can generate embeddings for unseen words
- **Example:** "playing" → ["pl", "la", "ay", "yi", "in", "ng"]

```
from gensim.models import FastText

sentences = [["natural", "language", "processing"]]
model = FastText(sentences, vector_size=100, window=5, min_count=1, min_n=3, max_n=6)
```

```
# Can handle OOV words
oov_vector = model.wv['unseen_word'] # Works due to subword info
```

Language Models

Q12: What is a language model and what are its types?

Answer:

Language Model: Statistical model that assigns probabilities to sequences of words.

Purpose:

- Predict next word in sequence
- Measure probability of sentence
- Generate text
- Speech recognition
- Machine translation

Types:

1. N-gram Models:

```
# Bigram example: P(word|previous_word)
# "I love" → P("natural"|"love"), P("language"|"natural")

from collections import defaultdict, Counter

def train_bigram_model(text):
    words = text.split()
    bigrams = [(words[i], words[i+1]) for i in range(len(words)-1)]

    bigram_counts = Counter(bigrams)
    word_counts = Counter(words[:-1]) # All words except last

    # P(w2|w1) = count(w1,w2) / count(w1)
    bigram_probs = {}
    for (w1, w2), count in bigram_counts.items():
        bigram_probs[(w1, w2)] = count / word_counts[w1]

    return bigram_probs
```

2. Neural Language Models:

- **RNN-based:** LSTM, GRU
- **Transformer-based:** GPT, BERT, T5

Evaluation Metrics:

- **Perplexity:** Lower is better, measures uncertainty
- **Cross-entropy:** Loss function for training

Perplexity Formula:

```
PP(W) = P(w1w2...wn)(-1/N)
Lower perplexity = Better model
```

Q13: Explain the concept of perplexity in language models.

Answer:

Perplexity: Measure of how well a language model predicts test data. Lower perplexity indicates better performance.

Mathematical Definition:

```
PP(W) = 2H(W)
where H(W) is cross-entropy
```

Intuitive Meaning:

- **Perplexity of 1:** Perfect prediction
- **Perplexity of N:** Model is as confused as random choice among N equally likely options
- **Perplexity of 100:** On average, model narrows down next word to ~100 candidates

Example:

```
import math

def calculate_perplexity(probabilities):
    """
    Calculate perplexity from list of word probabilities
    """
    log_sum = sum(math.log2(p) for p in probabilities)
    entropy = -log_sum / len(probabilities)
    perplexity = 2 ** entropy
    return perplexity

# Example: Model predictions for sentence "I love NLP"
word_probs = [0.1, 0.05, 0.2] # P("I"), P("love"|"I"), P("NLP"|"love")
pp = calculate_perplexity(word_probs)
print(f"Perplexity: {pp:.2f}")
```

Factors Affecting Perplexity:

- **Vocabulary Size:** Larger vocab generally increases perplexity
- **Training Data:** More data usually decreases perplexity
- **Model Complexity:** Better models have lower perplexity

- **Domain Match:** In-domain test data has lower perplexity

Named Entity Recognition (NER)

Q14: What is Named Entity Recognition and what are its applications?

Answer:

Named Entity Recognition (NER): Task of identifying and classifying named entities in text into predefined categories.

Common Entity Types:

- **PERSON:** Names of people
- **ORGANIZATION:** Companies, agencies, institutions
- **LOCATION:** Countries, cities, addresses
- **DATE:** Dates and times
- **MONEY:** Monetary values
- **PERCENT:** Percentage values
- **FACILITY:** Buildings, airports, highways

Example:

Text: "Apple Inc. was founded by Steve Jobs in Cupertino in 1976."

Entities:

- Apple Inc. → ORGANIZATION
- Steve Jobs → PERSON
- Cupertino → LOCATION
- 1976 → DATE

Implementation with spaCy:

```
import spacy

nlp = spacy.load("en_core_web_sm")
text = "Apple Inc. was founded by Steve Jobs in Cupertino in 1976."

doc = nlp(text)
for ent in doc.ents:
    print(f"{ent.text} → {ent.label_}")

# Output:
# Apple Inc. → ORG
# Steve Jobs → PERSON
# Cupertino → GPE
# 1976 → DATE
```

Applications:

- **Information Extraction:** Extract structured data from unstructured text
- **Knowledge Graphs:** Build relationships between entities
- **Question Answering:** Identify relevant entities in questions
- **Content Recommendation:** Tag content with relevant entities
- **Compliance:** Identify sensitive information (PII, PHI)

Q15: How do you evaluate NER systems?

Answer:

Evaluation Metrics:

1. Token-level Evaluation:

- **Precision:** $\text{Correct entity tokens} / \text{Predicted entity tokens}$
- **Recall:** $\text{Correct entity tokens} / \text{Actual entity tokens}$
- **F1-score:** Harmonic mean of precision and recall

2. Entity-level Evaluation:

- **Exact Match:** Entity boundaries and types must match exactly
- **Partial Match:** Allow partial overlap between predicted and true entities

Example:

```
True: "New York City" → LOCATION
Pred: "New York" → LOCATION

Token-level: 2/3 correct (66.7%)
Entity-level: 0/1 correct (0%) - not exact match
```

Implementation:

```
from seqeval.metrics import classification_report, f1_score

# True labels (BIO format)
y_true = [['B-PER', 'I-PER', 'O', 'B-LOC']]
# Predicted labels
y_pred = [['B-PER', 'I-PER', 'O', 'O']]

# Entity-level F1 score
f1 = f1_score(y_true, y_pred)
print(f"Entity-level F1: {f1}")

# Detailed report
report = classification_report(y_true, y_pred)
print(report)
```

BIO Tagging Scheme:

- **B-**: Beginning of entity
- **I-**: Inside entity (continuation)
- **O**: Outside entity (not an entity)

Sentiment Analysis

Q16: What is sentiment analysis and what are the different approaches?

Answer:

Sentiment Analysis: Task of determining emotional tone or opinion in text (positive, negative, neutral).

Approaches:

1. Lexicon-based:

- Use predefined dictionaries of sentiment words
- Simple aggregation of word sentiments

```
# VADER sentiment (lexicon-based)
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()
text = "This movie is really great!"
scores = analyzer.polarity_scores(text)
print(scores)
# {'neg': 0.0, 'neu': 0.294, 'pos': 0.706, 'compound': 0.6588}
```

2. Machine Learning:

- Train classifiers on labeled sentiment data
- Feature engineering (BoW, TF-IDF, n-grams)

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

# Example workflow
texts = ["I love this!", "This is terrible", "It's okay"]
labels = [1, 0, 1] # 1=positive, 0=negative

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(texts)

model = LogisticRegression()
model.fit(X, labels)
```

3. Deep Learning:

- Neural networks with word embeddings
- RNNs, CNNs, Transformers

Challenges:

- **Sarcasm:** "Great job!" (could be sarcastic)
- **Context Dependency:** Domain-specific sentiments
- **Negation:** "not good" vs "good"
- **Intensity:** "good" vs "excellent"

Q17: How do you handle negation in sentiment analysis?

Answer:

Negation Handling: Critical for accurate sentiment analysis as it can flip sentiment polarity.

Approaches:

1. Negation Scope Detection:

```
def handle_negation(tokens):
    negation_words = {'not', 'no', 'never', 'none', 'nobody', 'nothing', 'neither', 'nowhere'}

    result = []
    negate = False

    for token in tokens:
        if token.lower() in negation_words:
            negate = True
        elif token in ['.', '!', '?', ';', ',', 'but', 'however']:
            negate = False  # End negation scope
        elif negate:
            result.append(f"NOT_{token}")  # Mark negated words
        else:
            result.append(token)

    return result

# Example
tokens = ['this', 'is', 'not', 'good', 'at', 'all']
negated = handle_negation(tokens)
print(negated)
# Output: ['this', 'is', 'NOT_good', 'NOT_at', 'NOT_all']
```

2. Dependency Parsing:

- Use syntactic relationships to identify negation scope
- More accurate than window-based approaches

```
import spacy

nlp = spacy.load("en_core_web_sm")
text = "The movie is not good but the acting is excellent"

doc = nlp(text)
for token in doc:
    if token.dep_ == 'neg': # Negation dependency
        print(f"Negation: {token.text} modifies {token.head.text}")
```

3. Attention Mechanisms:

- Let model learn to focus on negation patterns
- Transformers naturally handle this through attention

Text Classification

Q18: Explain text classification and its common algorithms.

Answer:

Text Classification: Task of assigning predefined categories/labels to text documents.

Types:

- **Binary:** Spam vs Not Spam
- **Multi-class:** News categories (Sports, Politics, Technology)
- **Multi-label:** Document can have multiple tags

Pipeline:

1. **Text Preprocessing:** Clean, tokenize, normalize
2. **Feature Extraction:** BoW, TF-IDF, embeddings
3. **Model Training:** Choose and train classifier
4. **Evaluation:** Test performance on unseen data

Common Algorithms:

1. Naive Bayes:

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import TfidfVectorizer

# Good for text classification, handles high dimensions well
vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(texts)
```

```
nb_model = MultinomialNB()
nb_model.fit(X, labels)
```

2. Support Vector Machines:

```
from sklearn.svm import SVC

# Effective for high-dimensional text data
svm_model = SVC(kernel='linear', C=1.0)
svm_model.fit(X, labels)
```

3. Deep Learning:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# Neural network approach
model = Sequential([
    Embedding(vocab_size, embedding_dim, input_length=max_length),
    LSTM(128, dropout=0.2, recurrent_dropout=0.2),
    Dense(num_classes, activation='softmax')
])
```

Feature Engineering:

- **N-grams:** Unigrams, bigrams, trigrams
- **Character-level:** Good for typos, informal text
- **Syntactic:** POS tags, dependency features
- **Semantic:** Word embeddings, topic models

Q19: How do you handle imbalanced datasets in text classification?

Answer:

Imbalanced Datasets: When some classes have significantly fewer examples than others.

Strategies:

1. Resampling Techniques:

```
from imblearn.over_sampling import RandomOverSampler, SMOTE
from imblearn.under_sampling import RandomUnderSampler

# Random oversampling
ros = RandomOverSampler(random_state=42)
X_resampled, y_resampled = ros.fit_resample(X, y)

# SMOTE (Synthetic Minority Oversampling)
```

```
smote = SMOTE(random_state=42)
X_smote, y_smote = smote.fit_resample(X, y)
```

2. Class Weights:

```
from sklearn.linear_model import LogisticRegression
from sklearn.utils.class_weight import compute_class_weight

# Compute balanced class weights
class_weights = compute_class_weight('balanced', classes=np.unique(y), y=y)
class_weight_dict = dict(zip(np.unique(y), class_weights))

# Use in model
model = LogisticRegression(class_weight='balanced')
# or
model = LogisticRegression(class_weight=class_weight_dict)
```

3. Evaluation Metrics:

```
from sklearn.metrics import classification_report, f1_score

# Don't use accuracy for imbalanced data
# Use precision, recall, F1-score per class
report = classification_report(y_true, y_pred, average=None)

# Macro average (unweighted mean)
f1_macro = f1_score(y_true, y_pred, average='macro')

# Weighted average (by class frequency)
f1_weighted = f1_score(y_true, y_pred, average='weighted')
```

4. Data Augmentation:

```
# Text augmentation techniques
def augment_text(text, method='synonym_replacement'):
    if method == 'synonym_replacement':
        # Replace words with synonyms
        pass
    elif method == 'back_translation':
        # Translate to another language and back
        pass
    elif method == 'paraphrasing':
        # Generate paraphrases using language models
        pass
    return augmented_text
```

Machine Translation

Q20: What are the main approaches to machine translation?

Answer:

Machine Translation: Automatic translation of text from one language to another.

Evolution of Approaches:

1. Rule-Based Machine Translation (RBMT):

- Hand-crafted linguistic rules
- Morphological, syntactic, and semantic analysis
- **Pros:** Transparent, predictable
- **Cons:** Expensive to build, limited coverage

2. Statistical Machine Translation (SMT):

- Learn translation probabilities from parallel corpora
- Phrase-based models
- **Formula:** $P(e|f) = P(f|e) \times P(e) / P(f)$
- **Pros:** Data-driven, better coverage
- **Cons:** Requires large parallel data

3. Neural Machine Translation (NMT):

- End-to-end neural networks
- Encoder-decoder architecture with attention
- **Pros:** Fluent translations, handles long sentences
- **Cons:** Black box, requires large datasets

Modern NMT Architecture:

```
# Conceptual Transformer-based NMT
class TransformerNMT:
    def __init__(self):
        self.encoder = TransformerEncoder()
        self.decoder = TransformerDecoder()

    def translate(self, source_sentence):
        # Encode source sentence
        encoder_output = self.encoder(source_sentence)

        # Decode target sentence
        target_sentence = self.decoder(encoder_output)

        return target_sentence
```

Evaluation Metrics:

- **BLEU:** Compares n-gram overlap with references
- **METEOR:** Considers synonyms and paraphrases
- **BERTScore:** Uses contextual embeddings
- **Human Evaluation:** Fluency and adequacy ratings

Q21: What is the attention mechanism and why is it important?

Answer:

Attention Mechanism: Allows model to focus on relevant parts of input when generating each output token.

Problem with Basic Encoder-Decoder:

- Fixed-size context vector becomes bottleneck
- Information loss for long sequences
- All source information compressed into single vector

Attention Solution:

```
# Conceptual attention mechanism
def attention(query, keys, values):
    # Calculate attention weights
    scores = torch.matmul(query, keys.transpose(-2, -1))
    weights = torch.softmax(scores, dim=-1)

    # Weighted sum of values
    output = torch.matmul(weights, values)
    return output, weights

# In translation:
# Query: Current decoder state
# Keys/Values: All encoder states
```

Benefits:

- **Dynamic Context:** Different context for each output
- **Long Sequences:** No information bottleneck
- **Interpretability:** Attention weights show alignments
- **Performance:** Significant improvement in quality

Types of Attention:

- **Additive (Bahdanau):** Uses feedforward network
- **Multiplicative (Luong):** Simpler dot product
- **Self-Attention:** Attention within same sequence

- **Multi-Head:** Multiple attention computations in parallel

Advanced NLP Concepts

Q22: What are RNNs and why are they suitable for NLP tasks?

Answer:

Recurrent Neural Networks (RNNs): Neural networks designed for sequential data with memory of previous inputs.

Architecture:

```
h_t = tanh(W_h * h_{t-1} + W_x * x_t + b)
y_t = W_y * h_t + b_y
```

Where:

- h_t : hidden state at time t
- x_t : input at time t
- W_h, W_x, W_y : weight matrices

Why Good for NLP:

- **Sequential Nature:** Text is inherently sequential
- **Variable Length:** Can handle different sentence lengths
- **Memory:** Maintains information from previous words
- **Context:** Each word is processed in context of previous words

RNN Variants:

1. Vanilla RNN:

- Simple recurrent connection
- **Problem:** Vanishing gradient for long sequences

2. LSTM (Long Short-Term Memory):

```
import torch
import torch.nn as nn

class LSTM(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, num_classes):
        super(LSTM, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        embedded = self.embedding(x)
        lstm_out, (hidden, cell) = self.lstm(embedded)
```

```
# Use last hidden state for classification
output = self.fc(hidden[-1])
return output
```

3. GRU (Gated Recurrent Unit):

- Simpler than LSTM, fewer parameters
- Combines forget and input gates

Limitations:

- **Sequential Processing:** Cannot parallelize training
- **Vanishing Gradients:** For very long sequences
- **Limited Long-term Memory:** Despite LSTM improvements

Q23: What are the limitations of traditional RNNs and how do Transformers address them?

Answer:

RNN Limitations:

1. Sequential Processing:

- Must process tokens one by one
- Cannot parallelize training
- Slow for long sequences

2. Vanishing Gradients:

- Difficulty learning long-range dependencies
- Information from early tokens gets lost

3. Limited Parallelization:

- Training time increases linearly with sequence length
- Inefficient use of modern hardware (GPUs)

Transformer Solutions:

1. Self-Attention:

- Direct connections between all positions
- Parallel computation of attention weights
- No sequential bottleneck

2. Positional Encoding:

- Explicit position information
- No recurrence needed for position awareness

```
def positional_encoding(seq_len, d_model):
    pos = torch.arange(seq_len).unsqueeze(1)
    div_term = torch.exp(torch.arange(0, d_model, 2) *
                          -(math.log(10000.0) / d_model))

    pe = torch.zeros(seq_len, d_model)
    pe[:, 0::2] = torch.sin(pos * div_term) # Even positions
    pe[:, 1::2] = torch.cos(pos * div_term) # Odd positions
    return pe
```

3. Multi-Head Attention:

- Multiple attention computations in parallel
- Capture different types of relationships

```
def multi_head_attention(query, key, value, num_heads):
    # Split into multiple heads
    q_heads = split_heads(query, num_heads)
    k_heads = split_heads(key, num_heads)
    v_heads = split_heads(value, num_heads)

    # Apply attention to each head
    attention_outputs = []
    for q, k, v in zip(q_heads, k_heads, v_heads):
        attn_output = attention(q, k, v)
        attention_outputs.append(attn_output)

    # Concatenate and project
    combined = torch.cat(attention_outputs, dim=-1)
    return linear_projection(combined)
```

Benefits:

- **Parallelization:** All positions processed simultaneously
- **Long-range Dependencies:** Direct connections between distant tokens
- **Scalability:** Efficient on modern hardware
- **Performance:** State-of-the-art results across NLP tasks

Transformer Architecture

Q24: Explain the Transformer architecture in detail.

Answer:

Transformer: Attention-based architecture that revolutionized NLP by eliminating recurrence.

Key Components:

1. Multi-Head Self-Attention:

```

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, query, key, value, mask=None):
        batch_size = query.size(0)

        # Linear transformations
        Q = self.W_q(query)
        K = self.W_k(key)
        V = self.W_v(value)

        # Reshape for multi-head
        Q = Q.view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        K = K.view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        V = V.view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)

        # Scaled dot-product attention
        attention_output = scaled_dot_product_attention(Q, K, V, mask)

        # Concatenate heads
        attention_output = attention_output.transpose(1, 2).contiguous().view(
            batch_size, -1, self.d_model)

        return self.W_o(attention_output)

```

2. Position-wise Feed-Forward Networks:

```

class PositionwiseFeedForward(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.w_1 = nn.Linear(d_model, d_ff)
        self.w_2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        return self.w_2(torch.relu(self.w_1(x)))

```

3. Layer Normalization and Residual Connections:

```

class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout):
        super().__init__()
        self.attention = MultiHeadAttention(d_model, num_heads)
        self.feed_forward = PositionwiseFeedForward(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model)

```

```

        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # Self-attention with residual connection
        attn_output = self.attention(x, x, x, mask)
        x = self.norm1(x + self.dropout(attn_output))

        # Feed-forward with residual connection
        ff_output = self.feed_forward(x)
        x = self.norm2(x + self.dropout(ff_output))

    return x

```

Architecture:

- **Encoder:** Stack of N=6 transformer blocks
- **Decoder:** Stack of N=6 blocks with additional encoder-decoder attention
- **Embeddings:** Input embeddings + positional encoding
- **Output:** Linear layer + softmax for next token prediction

Q25: What is self-attention and how does it work?

Answer:

Self-Attention: Mechanism where each position in sequence attends to all positions in same sequence.

Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Where:

- Q (Query): What we're looking for
- K (Key): What to compare against
- V (Value): What to retrieve
- d_k : Dimension of key vectors (for scaling)

Step-by-Step Process:

1. Linear Transformations:

```

# Input sentence: "The cat sat on the mat"
# Each word becomes query, key, and value vectors

def create_qkv_vectors(embeddings, W_q, W_k, W_v):
    queries = torch.matmul(embeddings, W_q)
    keys = torch.matmul(embeddings, W_k)
    values = torch.matmul(embeddings, W_v)
    return queries, keys, values

```

2. Compute Attention Scores:

```
def compute_attention_scores(queries, keys):  
    # Dot product of queries and keys  
    scores = torch.matmul(queries, keys.transpose(-2, -1))  
  
    # Scale by square root of dimension  
    d_k = queries.size(-1)  
    scaled_scores = scores / math.sqrt(d_k)  
  
    return scaled_scores
```

3. Apply Softmax:

```
def apply_softmax(scores):  
    # Convert scores to probabilities  
    attention_weights = torch.softmax(scores, dim=-1)  
    return attention_weights
```

4. Weighted Sum of Values:

```
def compute_output(attention_weights, values):  
    # Multiply weights by values and sum  
    output = torch.matmul(attention_weights, values)  
    return output
```

Example:

```
Input: "The cat sat"  
For word "cat":  
- Query: Looking for what relates to "cat"  
- Attends to: "The" (0.1), "cat" (0.7), "sat" (0.2)  
- Output: Weighted combination of value vectors
```

Benefits:

- **Parallel Computation:** All positions processed simultaneously
- **Long-range Dependencies:** Direct connections between distant words
- **Interpretability:** Attention weights show word relationships
- **Flexibility:** Can capture various linguistic phenomena

BERT and Modern Models

Q26: What is BERT and how does it differ from previous models?

Answer:

BERT (Bidirectional Encoder Representations from Transformers): Pre-trained language model that processes text bidirectionally.

Key Innovations:

1. Bidirectional Context:

- Traditional models: Left-to-right (GPT) or right-to-left
- BERT: Processes both directions simultaneously
- Sees complete context for each word

2. Masked Language Modeling (MLM):

```
# Pre-training objective
input_text = "The [MASK] sat on the mat"
# Model learns to predict: "cat", "dog", etc.

# 15% of tokens are masked:
# 80% replaced with [MASK]
# 10% replaced with random token
# 10% kept unchanged (but still predicted)
```

3. Next Sentence Prediction (NSP):

```
# Binary classification task
sentence_A = "The man went to the store."
sentence_B = "He bought some milk." # IsNext = True

sentence_A = "The man went to the store."
sentence_B = "Penguins are flightless birds." # IsNext = False
```

Architecture Comparison:

Aspect	BERT	GPT	ELMo
Direction	Bidirectional	Left-to-right	Bidirectional (shallow)
Architecture	Transformer Encoder	Transformer Decoder	LSTM
Pre-training	MLM + NSP	Language Modeling	Language Modeling
Fine-tuning	Task-specific head	Few-shot/prompt	Feature extraction

BERT Variants:

- **BERT-Base:** 12 layers, 768 hidden, 12 heads, 110M parameters

- **BERT-Large:** 24 layers, 1024 hidden, 16 heads, 340M parameters
- **RoBERTa:** Optimized training (no NSP, larger batches)
- **ALBERT:** Parameter sharing, factorized embeddings
- **DistilBERT:** Smaller, faster version

Usage:

```
from transformers import BertTokenizer, BertModel
import torch

# Load pre-trained BERT
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

# Tokenize input
text = "BERT is a powerful NLP model"
inputs = tokenizer(text, return_tensors='pt')

# Get contextualized embeddings
with torch.no_grad():
    outputs = model(**inputs)
    embeddings = outputs.last_hidden_state

print(f"Shape: {embeddings.shape}") # [1, seq_len, 768]
```

Q27: How do you fine-tune BERT for downstream tasks?

Answer:

Fine-tuning: Adapting pre-trained BERT to specific tasks by training on task-specific data.

Fine-tuning Approaches:

1. Classification Tasks:

```
from transformers import BertForSequenceClassification, BertTokenizer
from torch.utils.data import DataLoader
import torch.nn as nn

# Load pre-trained model with classification head
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2 # Binary classification
)
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Prepare data
def prepare_data(texts, labels, tokenizer, max_length=512):
    encodings = tokenizer(
        texts,
        truncation=True,
        padding=True,
```

```

        max_length=max_length,
        return_tensors='pt'
    )
    return encodings, torch.tensor(labels)

# Training loop
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
model.train()

for batch in dataloader:
    optimizer.zero_grad()

    outputs = model(
        input_ids=batch['input_ids'],
        attention_mask=batch['attention_mask'],
        labels=batch['labels']
    )

    loss = outputs.loss
    loss.backward()
    optimizer.step()

```

2. Named Entity Recognition:

```

from transformers import BertForTokenClassification

# Token-level classification
model = BertForTokenClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=9 # BIO tags for 4 entity types
)

# Label mapping
label_map = {
    'O': 0, 'B-PER': 1, 'I-PER': 2,
    'B-LOC': 3, 'I-LOC': 4, 'B-ORG': 5, 'I-ORG': 6,
    'B-MISC': 7, 'I-MISC': 8
}

```

3. Question Answering:

```

from transformers import BertForQuestionAnswering

# Span prediction for extractive QA
model = BertForQuestionAnswering.from_pretrained('bert-base-uncased')

# Input format: [CLS] question [SEP] context [SEP]
# Output: Start and end positions of answer span

```

Fine-tuning Best Practices:

1. Learning Rate:

```
# Lower learning rates for pre-trained layers
optimizer = torch.optim.AdamW([
    {'params': model.bert.parameters(), 'lr': 2e-5},
    {'params': model.classifier.parameters(), 'lr': 2e-4}
])
```

2. Gradual Unfreezing:

```
# Start by training only classifier
for param in model.bert.parameters():
    param.requires_grad = False

# After few epochs, unfreeze BERT layers
for param in model.bert.parameters():
    param.requires_grad = True
```

3. Layer-wise Learning Rate Decay:

```
# Different learning rates for different layers
def get_layer_wise_lr(layer_num, base_lr=2e-5, decay_rate=0.95):
    return base_lr * (decay_rate ** layer_num)
```

Evaluation Metrics

Q28: What are the common evaluation metrics for NLP tasks?

Answer:

Evaluation Metrics by Task:

1. Classification Metrics:

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, classification_report

def evaluate_classification(y_true, y_pred):
    accuracy = accuracy_score(y_true, y_pred)
    precision, recall, f1, support = precision_recall_fscore_support(y_true, y_pred, average='micro')

    print(f"Accuracy: {accuracy:.3f}")
    print(f"Precision: {precision:.3f}")
    print(f"Recall: {recall:.3f}")
    print(f"F1-Score: {f1:.3f}")

    return classification_report(y_true, y_pred)
```

2. Sequence Labeling (NER, POS):

```

from sequeval.metrics import f1_score, classification_report

# Entity-level evaluation
y_true = [['B-PER', 'I-PER', 'O', 'B-LOC']]
y_pred = [['B-PER', 'I-PER', 'O', 'O']]

f1 = f1_score(y_true, y_pred)
report = classification_report(y_true, y_pred)

```

3. Language Generation:

```

# BLEU Score for Machine Translation
from nltk.translate.bleu_score import sentence_bleu

reference = [['this', 'is', 'a', 'test']]
candidate = ['this', 'is', 'test']

bleu_score = sentence_bleu(reference, candidate)

# ROUGE Score for Summarization
from rouge import Rouge

rouge = Rouge()
scores = rouge.get_scores("generated summary", "reference summary")

```

4. Information Retrieval:

```

# Precision@K, Recall@K, MAP, MRR
def precision_at_k(relevant, retrieved, k):
    retrieved_k = retrieved[:k]
    relevant_retrieved = len(set(relevant) & set(retrieved_k))
    return relevant_retrieved / k

def average_precision(relevant, retrieved):
    ap = 0.0
    relevant_count = 0

    for i, item in enumerate(retrieved):
        if item in relevant:
            relevant_count += 1
            ap += relevant_count / (i + 1)

    return ap / len(relevant) if relevant else 0.0

```

Metric Selection Guidelines:

Task	Primary Metrics	Considerations
Binary Classification	Accuracy, F1, AUC-ROC	Use F1 for imbalanced data
Multi-class	Macro/Micro F1, Per-class metrics	Macro for balanced evaluation

Task	Primary Metrics	Considerations
NER	Entity-level F1, Exact match	sequeval for proper evaluation
Summarization	ROUGE-1, ROUGE-2, ROUGE-L	Human evaluation important
Translation	BLEU, METEOR, BERTScore	Multiple references better
QA	Exact Match, F1 (token overlap)	Consider partial credit

Q29: Explain BLEU score and its limitations.

Answer:

BLEU (Bilingual Evaluation Understudy): Measures n-gram overlap between machine translation and reference translations.

Formula:

$$\text{BLEU} = \text{BP} \times \exp(\sum(w_n \times \log(p_n)))$$

Where:

- BP: Brevity penalty
- p_n: Precision of n-grams
- w_n: Weights (typically uniform)

Calculation Steps:

1. N-gram Precision:

```
def calculate_ngram_precision(candidate, reference, n):
    candidate_ngrams = get_ngrams(candidate, n)
    reference_ngrams = get_ngrams(reference, n)

    overlap = 0
    for ngram in candidate_ngrams:
        if ngram in reference_ngrams:
            overlap += min(candidate_ngrams[ngram], reference_ngrams[ngram])

    total_candidate_ngrams = sum(candidate_ngrams.values())
    return overlap / total_candidate_ngrams if total_candidate_ngrams > 0 else 0
```

2. Brevity Penalty:

```
def brevity_penalty(candidate_length, reference_length):
    if candidate_length > reference_length:
        return 1.0
    else:
        return math.exp(1 - reference_length / candidate_length)
```

3. Final BLEU Score:

```

from nltk.translate.bleu_score import sentence_bleu, corpus_bleu

# Single sentence
reference = [['the', 'cat', 'is', 'on', 'the', 'mat']]
candidate = ['the', 'cat', 'sits', 'on', 'the', 'mat']

bleu_score = sentence_bleu(reference, candidate)
print(f"BLEU Score: {bleu_score:.4f}")

# Multiple references
references = [
    ['the', 'cat', 'is', 'on', 'the', 'mat'],
    ['there', 'is', 'a', 'cat', 'on', 'the', 'mat']
]
bleu_score = sentence_bleu([references], candidate)

```

BLEU Limitations:

1. Only Measures Precision:

- Ignores recall (coverage of reference)
- Favors shorter translations

2. No Semantic Understanding:

- "The cat is happy" vs "The feline is joyful" (same meaning, low BLEU)
- Word order changes penalized heavily

3. Requires Multiple References:

- Single reference may not capture all valid translations
- Expensive to create multiple references

4. Corpus-level vs Sentence-level:

- More reliable at corpus level
- Sentence-level scores can be misleading

Alternatives:

- **METEOR**: Considers synonyms and paraphrases
- **BERTScore**: Uses contextual embeddings
- **BLEURT**: Learned metric based on BERT
- **Human Evaluation**: Gold standard but expensive

Practical Implementation

Q30: How do you build an end-to-end NLP pipeline?

Answer:

End-to-End NLP Pipeline: Complete workflow from raw text to deployed model.

Pipeline Components:

1. Data Collection and Preparation:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# Load and explore data
def load_data(file_path):
    df = pd.read_csv(file_path)
    print(f"Dataset shape: {df.shape}")
    print(f"Columns: {df.columns.tolist()}")
    print(f"Missing values: {df.isnull().sum()}")
    return df

# Split data
def prepare_splits(texts, labels, test_size=0.2, random_state=42):
    X_train, X_temp, y_train, y_temp = train_test_split(
        texts, labels, test_size=test_size*2, random_state=random_state)
    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp, test_size=0.5, random_state=random_state)
    return X_train, X_val, X_test, y_train, y_val, y_test
```

2. Text Preprocessing:

```
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

class TextPreprocessor:
    def __init__(self, remove_stopwords=True, lemmatize=True):
        self.remove_stopwords = remove_stopwords
        self.lemmatize = lemmatize
        self.stop_words = set(stopwords.words('english'))
        self.lemmatizer = WordNetLemmatizer()

    def clean_text(self, text):
        # Convert to lowercase
        text = text.lower()

        # Remove special characters and digits
        text = re.sub(r'[^a-zA-Z\s]', '', text)
```

```

        # Remove extra whitespace
        text = ' '.join(text.split())

        return text

def preprocess(self, text):
    # Clean text
    text = self.clean_text(text)

    # Tokenize
    tokens = word_tokenize(text)

    # Remove stopwords
    if self.remove_stopwords:
        tokens = [token for token in tokens if token not in self.stop_words]

    # Lemmatize
    if self.lemmatize:
        tokens = [self.lemmatizer.lemmatize(token) for token in tokens]

    return ' '.join(tokens)

def preprocess_batch(self, texts):
    return [self.preprocess(text) for text in texts]

```

3. Feature Engineering:

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

class FeatureExtractor:
    def __init__(self, max_features=10000, ngram_range=(1, 2)):
        self.vectorizer = TfidfVectorizer(
            max_features=max_features,
            ngram_range=ngram_range,
            stop_words='english'
        )

    def fit_transform(self, texts):
        return self.vectorizer.fit_transform(texts)

    def transform(self, texts):
        return self.vectorizer.transform(texts)

    def get_feature_names(self):
        return self.vectorizer.get_feature_names_out()

```

4. Model Training and Evaluation:

```

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

```

```

import joblib

class NLPClassifier:
    def __init__(self, model_type='logistic'):
        if model_type == 'logistic':
            self.model = LogisticRegression(max_iter=1000)
        elif model_type == 'random_forest':
            self.model = RandomForestClassifier(n_estimators=100)

        self.preprocessor = TextPreprocessor()
        self.feature_extractor = FeatureExtractor()

    def fit(self, X_train, y_train):
        # Preprocess text
        X_processed = self.preprocessor.preprocess_batch(X_train)

        # Extract features
        X_features = self.feature_extractor.fit_transform(X_processed)

        # Train model
        self.model.fit(X_features, y_train)

    def predict(self, X_test):
        # Preprocess and extract features
        X_processed = self.preprocessor.preprocess_batch(X_test)
        X_features = self.feature_extractor.transform(X_processed)

        return self.model.predict(X_features)

    def evaluate(self, X_test, y_test):
        y_pred = self.predict(X_test)

        print("Classification Report:")
        print(classification_report(y_test, y_pred))

        print("\nConfusion Matrix:")
        print(confusion_matrix(y_test, y_pred))

        return y_pred

    def save_model(self, filepath):
        model_data = {
            'model': self.model,
            'preprocessor': self.preprocessor,
            'feature_extractor': self.feature_extractor
        }
        joblib.dump(model_data, filepath)

    def load_model(self, filepath):
        model_data = joblib.load(filepath)
        self.model = model_data['model']
        self.preprocessor = model_data['preprocessor']
        self.feature_extractor = model_data['feature_extractor']

```

5. Complete Pipeline:

```

def run_nlp_pipeline(data_path, text_column, label_column):
    # 1. Load data
    df = load_data(data_path)
    texts = df[text_column].tolist()
    labels = df[label_column].tolist()

    # 2. Split data
    X_train, X_val, X_test, y_train, y_val, y_test = prepare_splits(texts, labels)

    # 3. Train model
    classifier = NLPClassifier(model_type='logistic')
    classifier.fit(X_train, y_train)

    # 4. Evaluate on validation set
    print("Validation Results:")
    y_val_pred = classifier.evaluate(X_val, y_val)

    # 5. Final evaluation on test set
    print("\nTest Results:")
    y_test_pred = classifier.evaluate(X_test, y_test)

    # 6. Save model
    classifier.save_model('nlp_model.pkl')

    return classifier

# Usage
classifier = run_nlp_pipeline('data.csv', 'text', 'sentiment')

```

6. Deployment:

```

from flask import Flask, request, jsonify

app = Flask(__name__)

# Load trained model
classifier = NLPClassifier()
classifier.load_model('nlp_model.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.json
        text = data['text']

        prediction = classifier.predict([text])[0]

        return jsonify({
            'text': text,
            'prediction': prediction,
            'status': 'success'
        })
    except Exception as e:

```

```
        return jsonify({
            'error': str(e),
            'status': 'error'
        }), 400

if __name__ == '__main__':
    app.run(debug=True)
```

Quick Reference Guide

Text Preprocessing Checklist

1. **Lowercase conversion**
2. **Remove special characters/numbers**
3. **Tokenization** (word/sentence)
4. **Remove stop words** (task-dependent)
5. **Stemming/Lemmatization**
6. **Handle contractions**
7. **Normalize whitespace**

Common Libraries

```
# Core NLP
import nltk
import spacy
import gensim

# Deep Learning
from transformers import BertTokenizer, BertModel
import torch
import tensorflow as tf

# Traditional ML
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Evaluation
from seqeval.metrics import f1_score # For NER
from rouge import Rouge # For summarization
from nltk.translate.bleu_score import sentence_bleu # For translation
```

Model Selection Guide

- **Classification:** BERT, RoBERTa, DistilBERT
- **NER:** BERT-NER, spaCy models, BiLSTM-CRF
- **Summarization:** T5, BART, Pegasus
- **Translation:** T5, mT5, MarianMT
- **Question Answering:** BERT-QA, RoBERTa-QA, T5

Interview Tips

1. **Understand the task** - Classification, generation, or sequence labeling?
2. **Ask about data** - Size, quality, labeling process
3. **Discuss preprocessing** - Show awareness of text cleaning steps
4. **Know evaluation metrics** - Match metrics to task type
5. **Consider deployment** - Latency, scalability, resource constraints
6. **Handle edge cases** - OOV words, different domains, multiple languages

This comprehensive NLP cheat sheet covers fundamental concepts through advanced techniques. Practice implementing these concepts and stay updated with latest developments in transformer models and large language models.